

An extended model-based characterization of fine-grained access control for SQL queries

Hoang Nguyen 
ETH Zurich
hoang.nguyen@inf.ethz.ch

Manuel Clavel 
Eastern International University
manuel.clavel@eiu.edu.vn



Abstract—In the context of a project in model-driven security that focuses on the development of model-driven techniques for building secure data-centric (web) applications, we extend, in three (inter-related) dimensions, a recently proposed model-based characterization of fine-grained access control (FGAC) authorization for SQL queries. First, we extend the FGAC policies’ underlying data models by considering association-classes. Secondly, we extend the FGAC policies’ security modeling language by considering, as protected resources, the classes and the (explicit and implicit) associations introduced by the association-classes. Thirdly, we extend the clauses that define whether a user is authorized by an FGAC policy to execute a SQL query, to cover the case of queries retrieving information related to the association-classes. To illustrate our extensions and to demonstrate their applicability, we provide examples of authorization decisions for SQL queries with respect to FGAC policies. The artifact comprising the implementation of this model-based characterization and an executable version of the example is available at <https://doi.org/10.5281/zenodo.8176237>.

Index Terms—Model-Driven Security, Fine-Grained Access Control, Secure Database Queries, SecureUML

I. INTRODUCTION

Software engineering is the science of engineering software systems. Over the years, programming languages and methodologies have been developed with the goal of enhancing the reliability, maintainability, and cost-effectiveness of software systems. One crucial aspect of well-engineered software systems is security.

The latest OWASP Top Ten Project identifies *broken access control* as the most prevalent type of vulnerability in web security [1]. In our opinion, this should not come as a surprise, given the current state of affairs. As it is known, a typical web application triggers actions that create, read, update, or delete data stored in a database. When the data is sensitive, the actions on the database should be controlled. Simple access control policies de-

pend only on the user’s credentials. In this case, it may be possible to formalize them *declaratively*, independently of the application’s business logic. In contrast, fine-grained access control (FGAC) policies depend not only on the user’s credentials but also on the satisfaction of constraints in the current state of the database. Currently, database management systems do not provide “native” support for FGAC policies (see [2] for a more in-depth discussion). As a consequence, these policies are often implemented *programmatically*, by encoding them at appropriate places in the application layer, in a mostly ad-hoc manner. This approach makes web applications’ access control mechanisms error-prone, hard to maintain, and easy to break.

One promising solution, in our opinion, is model-driven security (MDS) [3]. MDS is a specialization of model-driven engineering for developing secure systems. More concretely, designers are supposed to specify system models along with their security requirements and then use transformation tools to automatically generate security-related system artifacts, such as access control infrastructures. Following MDS, [4] has recently proposed a model-based characterization of FGAC authorization policies for SQL queries. More specifically, it has proposed a set of clauses that formally define, based on the given data and security models, whether a user is authorized by an FGAC policy to execute a SQL query on a database. It is worth noting that, in contrast with previous MDS proposals [5], [4] considers that, for the sake of efficiency, security, and maintainability, FGAC policies for data access should be enforced at the data layer instead of at the application layer.

As expected from a model-driven approach, [4] builds its proposal upon well-defined models: namely, data models (a simplified version of UML class diagrams) and security models (a “dialect” of SecureUML [6]). A key design-feature of this proposal is that, for each data-

centric (web) application, the security model is tightly connected with the data model. More specifically: (i) the resources protected by the FGAC policies in the security model are specified in the data model; and (ii) the authorization constraints that specify the FGAC policies in the security model are OCL expressions whose contextual model is the data model. Consequently, if the expressiveness of the data modeling language is extended, then the expressiveness of the security modeling language will also be extended.

Contributions: In this paper, we propose to extend [4] in three (inter-related) dimensions. First, we extend the FGAC policies' underlying data models by considering *association-classes* as part of the data modeling language (Section III). Secondly, we extend the FGAC policies' security modeling language by considering, as protected resources, the classes and the (implicit and explicit) associations introduced by the association-classes (Section IV). Thirdly, we extend the clauses that define whether a user is authorized by an FGAC policy to execute a SQL query, to cover the case of queries retrieving information related to the association-classes (Section V).

To illustrate our extensions and to demonstrate their applicability, we introduce a simple, but non-trivial scenario: an application for managing elections-related sensitive information (Section II). This scenario shows that, by extending the data modeling language with association-classes, we are able to express FGAC policies that could not be expressed in [4] in a similarly concise and natural way. The main reason for this is the following. In a data model, association-classes are resources that “glue” together, in a precise and meaningful way, other resources. Thus, when specifying FGAC policies in a security model, in the case of an association-class, the corresponding OCL authorization constraint can contain references — in a concise and natural way — to the resources that are “glued” by the given association-class.

We conclude the paper with a discussion on related and future work (Section VI).

II. SCENARIO

Consider a basic application for managing elections-related sensitive information. In particular, our application manages information about voters, elections, and votes cast by voters in elections. Let us suppose that our application uses a SQL database, with three tables: *Voter* (to store information about voters), *Election* (to store information about elections), and *Vote* (to store information about votes cast by voters in elections). The

1	<code>SELECT value FROM Vote</code>
2	<code>SELECT value FROM Vote WHERE voters = 'Alice'</code>
3	<code>SELECT elections FROM Vote WHERE voters = 'Bob'</code>
4	<code>SELECT value FROM Vote WHERE elections = 'Election_2023'</code>
5	<code>SELECT value FROM Vote JOIN (SELECT Voter_id FROM Voter WHERE Voter_id = 'Bob') AS TEMP ON voters = TEMP.Voter_id</code>

Fig. 1. Some SQL queries about elections, voters, and votes.

table *Voter* has a column *Voter_id* (its primary key) and a column *name* (to store the name of the voter). The table *Election* has a column *Election_id* (its primary key) and a column *description* (to store the description of the election). Finally, the table *Vote* has a column *Vote_id* (its primary key), a column *value* (to store the content of the vote), a column *voters* (to store a reference, foreign key, to the voter who cast the vote), and a column *elections* (to store a reference, foreign key, to the election in which the vote was cast).

Our sample application must enforce the following fine-grained access control policy:

- For each election, everybody can know the voters who participated in the election.
- For each election, everybody can know the votes that were cast in the election.
- For each vote, only the voter who cast the vote can know who cast it.
- For each vote, everybody can know the content/value of the vote.

Consider now the following questions about the SQL queries shown in Figure 1:

- Can Alice execute query #1 in every scenario?
- Can Alice execute query #2 in every scenario?
What about Bob?
- Can Bob execute query #2 in a scenario where Alice has not cast any vote yet?
- Can Bob execute query #3 in every scenario?
What about Alice?
- Can Alice execute query #4 in every scenario?
What about Bob?
- Can Bob execute query #5 in every scenario?
What about Alice?

The key contribution of [4] was to propose a formal characterization of FGAC for SQL queries that provide answers to questions as the above regarding FGAC au-

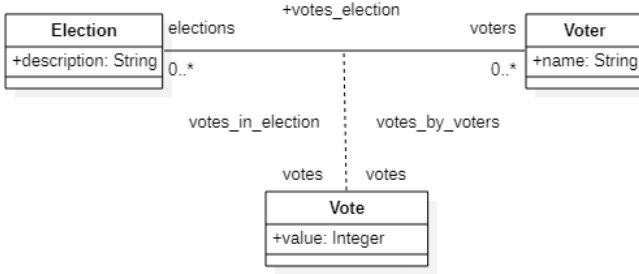


Fig. 2. A UML-like diagram representing the data model EM.

thorization for SQL queries. However, due to the limited expressiveness of its modeling languages, the proposal in [4] cannot cover — in a concise and natural way — the scenario discussed above.

III. MODELING DATA

In this section we define our notions of data and object models. We extend the corresponding notions in [4] by considering also association-classes as part of the data modeling language.

Definition 1. Let $\mathcal{T}$ be a set of predefined types. A data model $\mathcal{D}$ is a tuple $\langle C, AS, ASC, AT \rangle$, where:

- C is a set of classes c .
- AS is a set of associations as , as is a tuple $\langle asi, ase_l, c_l, ase_r, c_r \rangle$ where asi is the association's identifier, ase_l and ase_r are the “left” and “right” association's ends, and c_l and c_r are the classes (including association-classes) of the objects at the corresponding association's ends.
- ASC is a set of association-classes asc , asc is a tuple $\langle c, as, asc_l, asce_l, asc_r, asce_r \rangle$ where c is the association-class's class, as is the association-class's association (defined as above), asc_l and asc_r are the identifiers of the (implicit) “left” and “right” associations between the association-class's class and the ends of the association-class's association, and $asce_l$ and $asce_r$ are the “left” and “right” ends of the (implicit) associations between the association-class's class and the ends of the association-class's association.
- AT is a set of attributes at , $at = \langle ati, c, t \rangle$, where ati is the attribute's identifier, c is the class (or association-class) of the attribute, and t is the type of the values of the attribute (t can be either a class, an association-class, or a predefined type in $\mathcal{T}$).

In what follows, we denote by $ASC|_{\text{class}}$ the set of classes introduced by the association-classes in ASC .

Using our notion of data model, we can model the data in our running example as a tuple EM, $EM = \langle C_{EM}, AS_{EM}, ASC_{EM}, AT_{EM} \rangle$, where

$$\begin{aligned}
 C_{EM} &= \{\text{Voter}, \text{Election}\} \\
 AS_{EM} &= \emptyset, \\
 ASC_{EM} &= \{ \langle \text{Vote}, \\
 &\quad \langle \text{votes_election}, \text{elections}, \text{Election}, \text{voters}, \text{Voter} \rangle, \\
 &\quad \langle \text{votes_in_election}, \text{votes}, \text{votes_by_voters}, \text{votes} \rangle \} \\
 AT_{EM} &= \{ \langle \text{name}, \text{Voter}, \text{String} \rangle, \\
 &\quad \langle \text{description}, \text{Election}, \text{String} \rangle, \\
 &\quad \langle \text{value}, \text{Vote}, \text{Integer} \rangle \}
 \end{aligned}$$

Figure 2 shows a graphical, UML-like representation of the data model EM.

Definition 2. Let $\mathcal{D} = \langle C, AS, ASC, AT \rangle$ be a data model. An object model $\mathcal{O}$ of $\mathcal{D}$ (also called an instance of $\mathcal{D}$) is a tuple $\langle OC, OAS, OASC, OAT \rangle$ where:

- OC is a set of objects o , $o = \langle \check{o}, c \rangle$, where $\check{o}$ is the object's identifier, c is the object's class, and $c \in (C \cup ASC|_{\text{class}})$.
- OAS is a set of association links asl , asl is a tuple $\langle as, \langle \check{o}_l, c_l \rangle, \langle \check{o}_r, c_r \rangle \rangle$, in which association $as = \langle asi, ase_l, c_l, ase_r, c_r \rangle$ is in AS , $\langle \check{o}_l, c_l \rangle \in OC$ and $\langle \check{o}_r, c_r \rangle \in OC$. The association link asl denotes that there is a link of the association as between the objects $\langle \check{o}_l, c_l \rangle$ and $\langle \check{o}_r, c_r \rangle$, where $\langle \check{o}_l, c_l \rangle$ is at the end ase_l and $\langle \check{o}_r, c_r \rangle$ is at the end ase_r .
- $OASC$ is a set of association links $ascl$, $ascl$ is a tuple $\langle asc, \langle \check{o}, c \rangle, \langle \check{o}_l, c_l \rangle, \langle \check{o}_r, c_r \rangle \rangle$, in which $asc = \langle c, as, asc_l, asce_l, asc_r, asce_r \rangle$ is an association-class in ASC , and $as = \langle asi, ase_l, c_l, ase_r, c_r \rangle$, is an association with objects $\langle \check{o}, c \rangle$, $\langle \check{o}_l, c_l \rangle$, and $\langle \check{o}_r, c_r \rangle \in OC$. The association link $ascl$ denotes that there is a link of the association-class asc between the objects $\langle \check{o}_l, c_l \rangle$ and $\langle \check{o}_r, c_r \rangle$, where $\langle \check{o}_l, c_l \rangle$ is at the end ase_l and $\langle \check{o}_r, c_r \rangle$ is at the end ase_r , and where $\langle \check{o}, c \rangle$ is the object associated to the association-class's link $ascl$.
- OAT is a set of attribute values atv , atv is a tuple $\langle \langle ati, c, t \rangle, \langle \check{o}, c \rangle, vl \rangle$ where $\langle ati, c, t \rangle \in AT$, $\langle \check{o}, c \rangle \in OC$, and vl is a value of the type t . The attribute value atv denotes that the object $\langle \check{o}, c \rangle$ has value vl for the attribute $\langle ati, c, t \rangle$.

Mapping data and object models to databases

Our model-based characterization of access control for SQL queries assumes that queries are executed on databases that implement the policies' underlying data models, as well as the object models of interest, according to the mappings defined below. We extend

the mappings defined in [4] by considering association-classes and instances of association-classes. Note that our mapping of association-classes into SQL is both intuitive and direct, which in turn has a positive impact in the applicability of our approach, as shown in our scenario.

Notation: Let $\mathcal{D} = \langle C, AS, ASC, AT \rangle$ be a data model. Let c be a class in C . In our mapping below, we denote by $\bar{c}$ the name of the SQL table that corresponds to the class c ; by convention $\bar{c}$ is the class's identifier c . Let as be an association in AS , with $as = \langle asi, ase_l, c_l, ase_r, c_r \rangle$. In our mapping below, we denote by $\overline{as}$ the name of the SQL table that corresponds to the association as ; by convention $\overline{as}$ is the association's identifier asi . Finally, let asc be an association-class in ASC , with $asc = \langle c, as, asc_l, asce_l, asc_r, asce_r \rangle$ and $as = \langle asi, ase_l, c_l, ase_r, c_r \rangle$. In our mapping below, we denote by $\overline{asc}$ the name of the SQL table that corresponds to the association-class asc ; by convention $\overline{asc}$ is the association-class's class's identifier c .

Definition 3. Let $\mathcal{D} = \langle C, AS, ASC, AT \rangle$ be a data model. Our mapping of $\mathcal{D}$ to SQL, denoted by $\overline{\mathcal{D}}$, is defined as follows:

- For every $c \in C$,
`CREATE TABLE  $\bar{c}$  ( $\bar{c\_id}$  varchar PRIMARY KEY);`
- For every association $as \in AS$, where as is a tuple $\langle asi, ase_l, c_l, ase_r, c_r \rangle$,
`CREATE TABLE $\overline{as}$ (
 ase_l varchar NOT NULL,
 ase_r varchar NOT NULL,
FOREIGN KEY $fk_c_l_ase_l$ (ase_l)
REFERENCES c_l (c_l_id),
FOREIGN KEY $fk_c_r_ase_r$ (ase_r)
REFERENCES c_r (c_r_id));`

Moreover,

`ALTER TABLE $\overline{as}$ ADD UNIQUE
unique_link (ase_l, ase_r);`

- For every association-class $asc \in ASC$, where asc is a tuple $\langle c, as, asc_l, asce_l, asc_r, asce_r \rangle$ and $as = \langle asi, ase_l, c_l, ase_r, c_r \rangle$,
`CREATE TABLE $\overline{asc}$ (
 $\overline{asc_id}$ varchar PRIMARY KEY,
 ase_l varchar NOT NULL,
 ase_r varchar NOT NULL,
FOREIGN KEY $fk_c_l_ase_l$ (ase_l)
REFERENCES c_l (c_l_id),
FOREIGN KEY $fk_c_r_ase_r$ (ase_r)
REFERENCES c_r (c_r_id));`

Moreover,

`ALTER TABLE $\overline{asc}$ ADD UNIQUE
unique_link (ase_l, ase_r);`

- For every attribute $at \in AT$, with $at = \langle ati, c, t \rangle$,
`ALTER TABLE  $\bar{c}$  ADD COLUMN  $ati$  SqlType( $t$ );`
where:
 - if $t = \text{Integer}$, then $\text{SqlType}(t) = \text{int}$.
 - if $t = \text{String}$, then $\text{SqlType}(t) = \text{varchar}$.
 - if $t \in (C \cup ASC|_{\text{class}})$,
then $\text{SqlType}(t) = \text{varchar}$.
Moreover, if $t \in (C \cup ASC|_{\text{class}})$, then
`ALTER TABLE $\bar{c}$ ADD FOREIGN KEY
 fk_c_ati (ati) REFERENCES $\bar{t}$ ($\bar{t_id}$);`

Definition 4. Let $\mathcal{D} = \langle C, AS, ASC, AT \rangle$ be a data model. Let $\mathcal{O} = \langle OC, OAS, OASC, OAT \rangle$ be an object model of $\mathcal{D}$. The mapping of $\mathcal{O}$ to SQL, denoted by $\overline{\mathcal{O}}$, is defined as follows:

- For every object $\langle \check{o}, c \rangle \in OC$,
`INSERT INTO  $\bar{c}$  ( $\bar{c\_id}$ ) VALUES ( $\check{o}$ );`
- For every link of association $asl \in OAS$, where asl is a tuple $\langle as, \langle \check{o}_l, c_l \rangle, \langle \check{o}_r, c_r \rangle \rangle$ with $as = \langle asi, ase_l, c_l, ase_r, c_r \rangle$,
`INSERT INTO  $\overline{as}$  ( $ase_l, ase_r$ ) VALUES ( $\check{o}_l, \check{o}_r$ );`
- For every link of association-class $ascl \in OASC$, where $ascl$ is a tuple $\langle asc, \langle \check{o}, c \rangle, \langle \check{o}_l, c_l \rangle, \langle \check{o}_r, c_r \rangle \rangle$, with $asc = \langle c, as, asc_l, asce_l, asc_r, asce_r \rangle$ and $as = \langle asi, ase_l, c_l, ase_r, c_r \rangle$,
`INSERT INTO $\overline{asc}$ ($\overline{asc_id}, ase_l, ase_r$)
VALUES ($\check{o}, \check{o}_l, \check{o}_r$);`
- For every attribute value $atv \in OAT$, with $atv = \langle \langle ati, c, t \rangle, \langle \check{o}, c \rangle, vl \rangle$,
`UPDATE  $\bar{c}$  SET  $ati$  = SqlValue( $vl$ ) WHERE  $\bar{c\_id} = \check{o}$ ;`
where:
 - if $t \in (C \cup ASC|_{\text{class}})$, then $\text{SqlValue}(vl) = \check{vl}$.
 - otherwise, $\text{SqlValue}(vl) = vl$.

Notation: Let $\mathcal{D}$ be a data model. Let $\mathcal{O}$ be an object model of $\mathcal{D}$. Let q be a SQL query on $\overline{\mathcal{D}}$. We denote by $\text{Exec}(\overline{\mathcal{O}}, q)$ the result of executing q in $\overline{\mathcal{O}}$ according to the semantics of SQL.

IV. MODELING FINE-GRAINED ACCESS CONTROL POLICIES

In this section we define our notion of security models. We extend the corresponding notion in [4] by considering as sensitive data the classes and the (implicit and explicit) associations introduced by the association-classes.

SecureUML [6] is our modeling language of choice for specifying access control policies on resources. In SecureUML, resources are protected by controlling the actions that provide access to them. However, SecureUML leaves open the nature of the protected resources and,

consequently, of the corresponding controlled actions. These are to be declared in a so-called SecureUML *dialect*. Next we define the read-actions (both abstract and concrete) that we consider in our SecureUML dialect:

Definition 5. Let $\mathcal{D} = \langle C, AS, ASC, AT \rangle$ be a data model. Then, we denote by $\text{Act}(\mathcal{D})$ the following set of (abstract) read-actions:

- For every association $as \in AS$, with $as = \langle asi, ase_l, c_l, ase_r, c_r \rangle$, the action $\text{read}(asi) \in \text{Act}(\mathcal{D})$.
- For every association-class $asc \in ASC$, with $asc = \langle c, as, asc_l, asce_l, asc_r, asce_r \rangle$ and $as = \langle asi, ase_l, c_l, ase_r, c_r \rangle$, the actions:
 - $\text{read}(asi) \in \text{Act}(\mathcal{D})$,
 - $\text{read}(asc_l) \in \text{Act}(\mathcal{D})$, and
 - $\text{read}(asc_r) \in \text{Act}(\mathcal{D})$.
- For every attribute $at = \langle ati, c, t \rangle$ in AT , the action $\text{read}(ati) \in \text{Act}(\mathcal{D})$.

Definition 6. Let $\mathcal{D} = \langle C, AT, AS, ASC \rangle$ be a data model. Let $\mathcal{O} = \langle OC, OAS, OASC, OAT \rangle$ be an instance of $\mathcal{D}$. Then, we denote by $\text{Act}(\mathcal{O})$ the following set of (concrete) instances of read-actions:

- For every association $as \in AS$, where as is a tuple $\langle asi, ase_l, c_l, ase_r, c_r \rangle$, and every pair of objects $\langle o_l, o_r \rangle \in OC$ such that $o_l = \langle \check{o}_l, c_l \rangle$ and $o_r = \langle \check{o}_r, c_r \rangle$, the action $\text{read}(asi, o_l, o_r) \in \text{Act}(\mathcal{O})$, which represents the action of reading/checking if there exists a link of the association as between o_l and o_r .
- For every association-class $asc \in ASC$, where asc is a tuple $\langle c, as, asc_l, asce_l, asc_r, asce_r \rangle$ with $as = \langle asi, ase_l, c_l, ase_r, c_r \rangle$, and every triple of objects $\langle o, o_l, o_r \rangle \in OC$ such that $o = \langle \check{o}, c \rangle$, $o_l = \langle \check{o}_l, c_l \rangle$, and $o_r = \langle \check{o}_r, c_r \rangle$, the following actions are in $\text{Act}(\mathcal{O})$:
 - $\text{read}(asi, o_l, o_r, o)$, which represents the action of reading/checking if there exists a link, with object o , of the association-class asc between o_l and o_r .
 - $\text{read}(asc_l, o_l, o)$, which represents the action of reading/checking if there exists a link of the (implicit) association asc_l , such that the object at the end ase_l is o_l and the object at the (implicit) end $asce_l$ is o .
 - $\text{read}(asc_r, o_r, o)$, which represents the action of reading/checking if there exists a link of the (implicit) association asc_r , such that the object at the end ase_r is o_r and the object at the (implicit) end $asce_r$ is o .

- For every attribute $at = \langle ati, c, t \rangle$, $at \in AT$, and every object o such that $o = \langle \check{o}, c \rangle$, the action $\text{read}(ati, o) \in \text{Act}(\mathcal{O})$, which represents the action of reading the value of the attribute at of o .

As a language for modeling access control policies, SecureUML is extension of Role-Based Access Control (RBAC) [7]. In RBAC, permissions are assigned to roles, and roles are assigned to users. However, in SecureUML, one can model access control decisions that depend on two kinds of information: namely, static information, i.e., the assignments of users and permissions to roles; and dynamic information, i.e., the satisfaction of *authorization constraints* in the current state of the system. Authorization constraints are specified in our SecureUML dialect using OCL [8]. More concretely, we consider authorization constraints whose satisfaction depend on information related to: (i) the object who is attempting to perform the read-action; (ii) the object whose attribute is attempted to be read; and, (iii) the objects between which the association's link is attempted to be read. By convention, we denote (i) by the keyword `caller`; we denote (ii) by the keyword `self`; and we denote (iii) by using as keywords the corresponding association-ends. Next we define the notion of security models in our dialect of SecureUML.

Definition 7. Let $\mathcal{D}$ be a data model. Then, a *security model* S for $\mathcal{D}$ is a tuple $S = \langle R, \text{auth} \rangle$, where R is a set of roles, and $\text{auth} : R \times \text{Act}(\mathcal{D}) \rightarrow \text{Exp}(\mathcal{D})$ is a function that assigns to each role $r \in R$ and each action $a \in \text{Act}(\mathcal{D})$ an authorization constraint $\text{exp} \in \text{Exp}(\mathcal{D})$.

Using our notion of security model, we can model the FGAC policy in our running example as a tuple EMS , $\text{EMS} = \langle R_{\text{EMS}}, \text{auth}_{\text{EMS}} \rangle$, where the underlying data model is EM (depicted in Figure 2), the set R_{EMS} of roles contains only one role `User` (representing any user of the application), and the function $\text{auth}_{\text{EMS}}()$ is defined as follows:

```
authEMS(User, read(voters_election)) = true,
authEMS(User, read(votes_in_election)) = true,
authEMS(User, read(votes_by_voters)) = (voters = caller),
authEMS(User, read(value)) = true.
```

Finally, we formalize the semantics of our security models by defining a predicate $\text{Auth}()$, which represents, for a given model, whether a user is authorized to execute a read-action on a given scenario.

Notation: Let $\mathcal{D}$ be a data model and $\mathcal{O}$ be an object model of $\mathcal{D}$. Let exp be an OCL expression in context of

$\mathcal{D}$. We denote by $\text{Eval}(\mathcal{O}, \text{exp})$ the result of evaluating exp in $\mathcal{O}$ according to the semantics of OCL.

Definition 8. Let $\mathcal{D}$ be a data model. Let $\mathcal{S} = \langle R, \text{auth} \rangle$ be a security model for $\mathcal{D}$. Let r be a role in R . Let $\mathcal{O} = \langle OC, OAS, OASC, OAT \rangle$ be an object model of $\mathcal{D}$. Let u be a user object in OC . Then, we define the predicate $\text{Auth}()$ as follows:

- For every association $as \in AS$, where as is a tuple $\langle asi, ase_l, c_l, ase_r, c_r \rangle$ and every action $\text{read}(asi, o_l, o_r)$ in $\text{Act}(\mathcal{O})$,
 $\text{Auth}(\mathcal{S}, \mathcal{O}, u, r, \text{read}(asi, o_l, o_r))$ holds
 $\iff \text{Eval}(\mathcal{O}, \sigma \cdot \text{auth}(r, \text{read}(asi))) = \text{true}$,
 where σ is the substitution function:
 $\sigma = [ase_l \mapsto o_l; ase_r \mapsto o_r; \text{caller} \mapsto u]$.
- For every association-class $asc \in ASC$, asc is a tuple $\langle c, as, asc_l, asc_l, asc_r, asc_r \rangle$ with $as = \langle asi, ase_l, c_l, ase_r, c_r \rangle$ then,
 - for every action $\text{read}(asi, o_l, o_r, o) \in \text{Act}(\mathcal{O})$,
 $\text{Auth}(\mathcal{S}, \mathcal{O}, u, r, \text{read}(asi, o_l, o_r, o))$ holds
 $\iff \text{Eval}(\mathcal{O}, \sigma \cdot \text{auth}(r, \text{read}(asi))) = \text{true}$,
 where σ is the substitution function
 $\sigma = [\text{self} \mapsto o, ase_l \mapsto o_l; ase_r \mapsto o_r; \text{caller} \mapsto u]$.
 - for every action $\text{read}(asc_l, o_l, o) \in \text{Act}(\mathcal{O})$,
 $\text{Auth}(\mathcal{S}, \mathcal{O}, u, r, \text{read}(asc_l, o_l, o))$ holds
 $\iff \text{Eval}(\mathcal{O}, \sigma \cdot \text{auth}(r, \text{read}(asc_l))) = \text{true}$,
 where σ is the substitution function
 $\sigma = [ase_l \mapsto o_l; asc_l \mapsto o; \text{caller} \mapsto u]$.
 - for every action $\text{read}(asc_r, o_r, o) \in \text{Act}(\mathcal{O})$,
 $\text{Auth}(\mathcal{S}, \mathcal{O}, u, r, \text{read}(asc_r, o_r, o))$ holds
 $\iff \text{Eval}(\mathcal{O}, \sigma \cdot \text{auth}(r, \text{read}(asc_r))) = \text{true}$,
 where σ is the substitution function
 $\sigma = [ase_r \mapsto o_r; asc_r \mapsto o; \text{caller} \mapsto u]$.
- For every attribute $at = \langle ati, c, t \rangle$, $at \in AT$, and every action $\text{read}(ati, o)$ in $\text{Act}(\mathcal{O})$,
 $\text{Auth}(\mathcal{S}, \mathcal{O}, u, r, \text{read}(ati, o))$ holds
 $\iff \text{Eval}(\mathcal{O}, \sigma \cdot \text{auth}(r, \text{read}(ati))) = \text{true}$,
 where σ is the substitution function
 $\sigma = [\text{self} \mapsto o; \text{caller} \mapsto u]$.

V. MODEL-BASED SQL QUERY AUTHORIZATION

In this section, we propose our model-based characterization of FGAC-authorization for SQL queries. We extend the definition of the predicate $\text{AuthQuery}()$ given in [4], to cover the case of SQL queries retrieving information related to the association-classes (namely, cases 3, 6, 7, 9, 10 below).

Preliminaries: The predicate $\text{AuthQuery}()$ specifies whether a user is authorized by an FGAC policy to execute a SQL query. The definition of the predicate $\text{AuthQuery}()$ takes into account an *attacker* that: (i) may know the application's database but cannot know the application's FGAC-policy; (ii) may know SQL and can attempt to execute SQL queries in the application's database. In this scenario, checking whether a user is authorized to execute a SQL query on a database implies much more than simply checking that the final result satisfies the given FGAC policy, since a clever attacker can devise a query such that the simple fact that a final result is obtained reveals already some additional information, which may be confidential. To confront this attack, the definition of the predicate $\text{AuthQuery}()$ checks that a user is authorized to read any information that is accessed when executing a query (in particular, information involved in subqueries, where-clauses, and on-clauses), and not only the information displayed in the final result. To this end, the definition of the predicate $\text{AuthQuery}()$ assumes an order of execution based on the syntactic presentation of the query, and not on its semantics (and, certainly, not on the query transformation produced by the SQL optimizer). Not surprisingly then, the predicate $\text{AuthQuery}()$ may return different results for queries that are semantically equivalent, but have different syntactic presentations.¹

Scope: Our extended definition of the predicate $\text{AuthQuery}()$ covers the following query “patterns”:

- **SELECT** *selitems* **FROM** $\bar{c}$ **WHERE** *exp*
 where c is a class.
- **SELECT** *selitems* **FROM** $\overline{as}$ **WHERE** *exp*
 where as is an association.
- **SELECT** *selitems* **FROM** $\overline{asc}$ **WHERE** *exp*
 where asc is an association-class.
- **SELECT** *selitems* **FROM** *subselect* **WHERE** *exp*
 where *subselect* is a (sub) select-statement.
- **SELECT** *selitems* **FROM** *fromitem*₁
JOIN *fromitem*₂ **ON** *exp* **WHERE** *exp'*
 where *fromitem*₁ (resp. *fromitem*₂) is either $\bar{c}$, where c is a class; or $\overline{as}$, where as is an association; or $\overline{asc}$, where asc is an association-class asc ; or a select-statement *subselect*. For the sake of space limitation, we will only consider below some of the combinations within this “pattern”.

¹An interesting challenge is then to “help” the developer to write SQL queries that may have more chances to be authorized by the predicate $\text{AuthQuery}()$: moving, for example, the where-clauses as “inside” the query as possible does certainly “help”.

Auxiliary functions: We use the following functions in the definition of the predicate AuthQuery():

- PropsInSel(*selitems*): the set of *properties* (i.e., attributes and association-ends) that appear in a list *selitems* of selected items.
- PropsInWhe(*exp*): the set of *properties* (i.e., attributes and association-ends) that appear in a where-expression *exp*.
- PropsInOn(*exp*): the set of *properties* (i.e., attributes and association-ends) that appear in an on-expression *exp*.

The AuthQuery() predicate

Let $\mathcal{D}$ be a data model, $\mathcal{D} = \langle C, AS, ASC, AT \rangle$. Let $\mathcal{O}$ be an object model of $\mathcal{D}$. Let $\mathcal{S} = \langle R, \text{auth} \rangle$ be a security model for $\mathcal{D}$. Let q be a SQL query in $\overline{\mathcal{D}}$. Let r be a role in R . Let u be a user. We define the predicate AuthQuery() as follows:

Case 1. $q = \text{SELECT } selitems \text{ FROM } \bar{c} \text{ WHERE } exp$ where c is a class in C . Then, AuthQuery($\mathcal{S}, \overline{\mathcal{O}}, u, r, q$) holds if and only if:

- **Clause 1.a.** For every object $\check{o} \in \text{Exec}(\overline{\mathcal{O}}, \text{SELECT } \bar{c_id} \text{ FROM } \bar{c})$, and for every attribute $at = \langle ati, c, t \rangle$, such that $ati \in \text{PropsInWhe}(exp)$, it holds that:
Auth($\mathcal{S}, \mathcal{O}, u, r, \text{read}(ati, o)$).
- **Clause 1.b.** For every object $\check{o} \in \text{Exec}(\overline{\mathcal{O}}, \text{SELECT } \bar{c_id} \text{ FROM } \bar{c} \text{ WHERE } exp)$, for every attribute $at = \langle ati, c, t \rangle$, such that $ati \in \text{PropsInSel}(selitems)$, it holds that:
Auth($\mathcal{S}, \mathcal{O}, u, r, \text{read}(ati, o)$).

Case 2. $q = \text{SELECT } selitems \text{ FROM } \overline{as} \text{ WHERE } exp$ where $as = \langle asi, ase_l, c_l, ase_r, c_r \rangle$ is an association in AS . Then, AuthQuery($\mathcal{S}, \overline{\mathcal{O}}, u, r, q$) holds if and only if:

- **Clause 2.a.** For every object pair $\langle \check{o}_l, \check{o}_r \rangle \in \text{Exec}(\overline{\mathcal{O}}, \text{SELECT } \bar{c_l_id}, \bar{c_r_id} \text{ FROM } \bar{c_l}, \bar{c_r} \text{ WHERE } exp)$, it holds that:
Auth($\mathcal{S}, \mathcal{O}, u, r, \text{read}(asi, o_l, o_r)$).

Case 3. $q = \text{SELECT } selitems \text{ FROM } \overline{asc} \text{ WHERE } exp$ where asc is an association-class's class asc in ASC , with $asc = \langle c, as, asc_l, asc_e_l, asc_r, asc_e_r \rangle$ and $as = \langle asi, ase_l, c_l, ase_r, c_r \rangle$. Then, AuthQuery($\mathcal{S}, \overline{\mathcal{O}}, u, r, q$) holds if and only if all the clauses below hold:

- **Clause 3.a.** For every object $\check{o} \in \text{Exec}(\overline{\mathcal{O}}, \text{SELECT } \overline{asc_id} \text{ FROM } \overline{asc})$, and for every attribute $at =$

$\langle ati, c, t \rangle$, such that $ati \in \text{PropsInWhe}(exp)$, it holds that:

Auth($\mathcal{S}, \mathcal{O}, u, r, \text{read}(ati, o)$).

- **Clause 3.b.** For every object $\check{o} \in \text{Exec}(\overline{\mathcal{O}}, \text{SELECT } \overline{asc_id} \text{ FROM } \overline{asc} \text{ WHERE } exp)$, and for every attribute $at = \langle ati, c, t \rangle$, such that $ati \in \text{PropsInSel}(selitems)$, it holds that:
Auth($\mathcal{S}, \mathcal{O}, u, r, \text{read}(ati, o)$).
- **Clause 3.c.** If either ase_l or ase_r (or both) appear in $\text{PropsInSel}(selitems) \cup \text{PropsInWhe}(exp)$, then, for every pair of object $\langle \check{o}_l, \check{o}_r \rangle \in \text{Exec}(\overline{\mathcal{O}}, \text{SELECT } \bar{c_l_id}, \bar{c_r_id} \text{ FROM } \bar{c_l}, \bar{c_r} \text{ WHERE } exp)$, it holds that:
Auth($\mathcal{S}, \mathcal{O}, u, r, \text{read}(asi, o_l, o_r)$).
- **Clause 3.d.** If $\overline{asc_id}$ or any attribute of the association-class's class c appears in $props = \text{PropsInSel}(selitems) \cup \text{PropsInWhe}(exp)$, then:
 - If $ase_l \in props$, then for every object pair $\langle \check{o}_l, \check{o} \rangle \in \text{Exec}(\overline{\mathcal{O}}, \text{SELECT } ase_l, \overline{asc_id} \text{ FROM } \overline{asc} \text{ WHERE } exp)$, it holds that:
Auth($\mathcal{S}, \mathcal{O}, u, r, \text{read}(asc_l, o_l, o)$).
 - If $ase_r \in props$, then for every object pair $\langle \check{o}_r, \check{o} \rangle \in \text{Exec}(\overline{\mathcal{O}}, \text{SELECT } ase_r, \overline{asc_id} \text{ FROM } \overline{asc} \text{ WHERE } exp)$, it holds that:
Auth($\mathcal{S}, \mathcal{O}, u, r, \text{read}(asc_r, o_r, o)$).

We can now give answers to the questions that we posed before:

- *Can Alice execute query #1 in every scenario?* Yes.
Explanation. This is Case 3. Recall that, in our example, $\text{auth}_{\text{EMS}}(\text{User}, \text{read}(\text{value})) = \text{true}$.
(i) Clause 3.a. It is not applicable.
(ii) Clause 3.b. For every scenario $\mathcal{O}$, and every Vote-object o , it holds that
Auth($\text{EMS}, \mathcal{O}, \text{'Alice'}, \text{User}, \text{read}(\text{value}, o)$),
because with $\sigma = [\text{self} \mapsto o; \text{caller} \mapsto \text{'Alice'}]$:
Eval($\mathcal{O}, \sigma \cdot \text{auth}_{\text{EMS}}(\text{User}, \text{read}(\text{value}))$) = true.
(iii) Clause 3.c. It is not applicable.
(iv) Clause 3.d. It is not applicable.
- *Can Alice execute query #2 in every scenario?* Yes.
Explanation. This is Case 3. Recall that, in our example, $\text{auth}_{\text{EMS}}(\text{User}, \text{read}(\text{value})) = \text{true}$.
(i) Clause 3.a. It is not applicable.
(ii) Clause 3.b. For every scenario $\mathcal{O}$, and every Vote-object o cast by 'Alice', it holds that
Auth($\text{EMS}, \mathcal{O}, \text{'Alice'}, \text{User}, \text{read}(\text{value}, o)$),
because with $\sigma = [\text{self} \mapsto o; \text{caller} \mapsto \text{'Alice'}]$:
Eval($\mathcal{O}, \sigma \cdot \text{auth}_{\text{EMS}}(\text{User}, \text{read}(\text{value}))$) = true.

(iii) Clause 3.c. Recall that, in our example,
 $\text{auth}_{\text{EMS}}(\text{User}, \text{read}(\text{voters_election})) = \text{true}$.
 Then, for every scenario $\mathcal{O}$, and every pair of
 Election-object o_l and Voter-object o_r , such
 that o_r is 'Alice', it holds that

$\text{Auth}(\text{EMS}, \mathcal{O}, \text{'Alice'}, \text{User},$
 $\text{read}(\text{voters_election}, o_l, o_r)),$
 because with $\sigma = [\text{elections} \mapsto o_l; \text{voters} \mapsto$
 $o_r; \text{caller} \mapsto \text{'Alice'}]:$

$\text{Eval}(\mathcal{O}, \sigma \cdot \text{auth}_{\text{EMS}}(\text{User},$
 $\text{read}(\text{voters_election}))) = \text{true}.$

(iv) Clause 3.d. Recall that, in our example,
 $\text{auth}_{\text{EMS}}(\text{User}, \text{read}(\text{votes_by_voters}))$
 $= (\text{voters} = \text{caller}).$

Then, for every scenario $\mathcal{O}$, and every triple of
 Election-object o_l , Voter-object o_r , and Vote-
 object o , such that o_r is 'Alice', it holds that

$\text{Auth}(\text{EMS}, \mathcal{O}, \text{'Alice'}, \text{User},$
 $\text{read}(\text{votes_by_voters}, o_r, o))$

because with $\sigma = [\text{voters} \mapsto o_r; \text{caller} \mapsto$
 $\text{'Alice'}]:$

$\text{Eval}(\mathcal{O}, \sigma \cdot \text{auth}_{\text{EMS}}(\text{User},$
 $\text{read}(\text{votes_by_voters}))) = \text{true}.$

- What about Bob? Can Bob execute query #2 in every scenario? No.

Explanation. In this case, Clause 3.d does not hold.
 Suppose an scenario with at least one Election-
 object o_l , two Voters-objects, namely, 'Alice' and
 'Bob', and one Vote-object o . Suppose that 'Alice'
 cast the vote o in the election o_l . Let 'Alice' be the
 object o_r . It does not hold that

$\text{Auth}(\text{EMS}, \mathcal{O}, \text{'Bob'}, \text{User},$
 $\text{read}(\text{votes_by_voters}, o_r, o))$

because with $\sigma = [\text{voters} \mapsto o_r; \text{caller} \mapsto$
 $\text{'Bob'}]:$

$\text{Eval}(\mathcal{O}, \sigma \cdot \text{auth}_{\text{EMS}}(\text{User},$
 $\text{read}(\text{votes_by_voters}))) = \text{false}.$

For the rest of the queries in our example, due to space
 limitation, we will only indicate below our answers, and
 leave the task of verifying them to the interested reader.

- Can Bob execute query #2 in a scenario where Alice has not cast any vote yet? Yes.
- Can Bob execute query #3 in every scenario? Yes. What about Alice? Yes.
- Can Alice execute query #4 in every scenario? Yes. What about Bob? Yes.

Case 4. $q = \text{SELECT } \text{selitems} \text{ FROM } \text{subselect} \text{ WHERE } \text{exp}$. Then, $\text{AuthQuery}(\mathcal{S}, \overline{\mathcal{O}}, u, r, q)$ holds if and only if $\text{AuthQuery}(\mathcal{S}, \overline{\mathcal{O}}, u, r, \text{subselect})$ holds.

Case 5. $q = \text{SELECT } \text{selitems} \text{ FROM } \overline{c} \text{ JOIN } \overline{as} \text{ ON } \text{exp}$, where c is a class in C and as is an association in AS , with $as = \langle \text{asi}, \text{ase}_l, c_l, \text{ase}_r, c_r \rangle$. Then, $\text{AuthQuery}(\mathcal{S}, \overline{\mathcal{O}}, u, r, q)$ holds if and only if:

- **Clause 5.a.** For every object $\check{o} \in \text{Exec}(\overline{\mathcal{O}}, \text{SELECT } \overline{c_id} \text{ FROM } \overline{c})$, and for every attribute $at = \langle \text{ati}, c, t \rangle$, such that $\text{ati} \in \text{PropsInOn}(\text{exp})$, it holds that:
 $\text{Auth}(\mathcal{S}, \mathcal{O}, u, r, \text{read}(\text{ati}, o)).$
- **Clause 5.b.** For every object $\check{o} \in \text{Exec}(\overline{\mathcal{O}}, \text{SELECT } \overline{c_id} \text{ FROM } \overline{c} \text{ JOIN } \overline{as} \text{ ON } \text{exp})$, and for every attribute $at = \langle \text{ati}, c, t \rangle$, such that $\text{ati} \in \text{PropsInSel}(\text{selitems})$, it holds that:
 $\text{Auth}(\mathcal{S}, \mathcal{O}, u, r, \text{read}(\text{ati}, o)).$
- **Clause 5.c.** For every object pair $\langle \check{o}_l, \check{o}_r \rangle \in \text{Exec}(\overline{\mathcal{O}}, \text{SELECT } \overline{c_l_id}, \overline{c_r_id} \text{ FROM } \overline{c_l}, \overline{c_r})$, it holds that:
 $\text{Auth}(\mathcal{S}, \mathcal{O}, u, r, \text{read}(\text{asi}, o_l, o_r)).$

Case 6. $q = \text{SELECT } \text{selitems} \text{ FROM } \overline{c} \text{ JOIN } \overline{asc} \text{ ON } \text{exp}$, where c is a class in C and asc is an association-class in ASC , where asc is a tuple of type $\langle c', as, \text{asc}_l, \text{asc}_e, \text{asc}_r, \text{asc}_t \rangle$, with $as = \langle \text{asi}, \text{ase}_l, c_l, \text{ase}_r, c_r \rangle$. Then, $\text{AuthQuery}(\mathcal{S}, \overline{\mathcal{O}}, u, r, q)$ holds if and only if:

- **Clause 6.a.** For every object $\check{o} \in \text{Exec}(\overline{\mathcal{O}}, \text{SELECT } \overline{c_id} \text{ FROM } \overline{c})$, and for every attribute $at = \langle \text{ati}, c, t \rangle$, such that $\text{ati} \in \text{PropsInOn}(\text{exp})$, it holds that:
 $\text{Auth}(\mathcal{S}, \mathcal{O}, u, r, \text{read}(\text{ati}, o)).$
- **Clause 6.b.** For every object $\check{o} \in \text{Exec}(\overline{\mathcal{O}}, \text{SELECT } \overline{asc_id} \text{ FROM } \overline{asc})$, and for every attribute $at = \langle \text{ati}, c', t \rangle$, such that $\text{ati} \in \text{PropsInOn}(\text{exp})$, it holds that:
 $\text{Auth}(\mathcal{S}, \mathcal{O}, u, r, \text{read}(\text{ati}, o)).$
- **Clause 6.c.** For every object $\check{o} \in \text{Exec}(\overline{\mathcal{O}}, \text{SELECT } \overline{c_id} \text{ FROM } \overline{c} \text{ JOIN } \overline{asc} \text{ ON } \text{exp})$, and for every attribute $at = \langle \text{ati}, c, t \rangle$, such that $\text{ati} \in \text{PropsInSel}(\text{selitems})$, it holds that:
 $\text{Auth}(\mathcal{S}, \mathcal{O}, u, r, \text{read}(\text{ati}, o)).$
- **Clause 6.d.** For every object $\check{o} \in \text{Exec}(\overline{\mathcal{O}}, \text{SELECT } \overline{asc_id} \text{ FROM } \overline{c} \text{ JOIN } \overline{asc} \text{ ON } \text{exp})$, and for every attribute $at = \langle \text{ati}, c', t \rangle$, such that $\text{ati} \in \text{PropsInSel}(\text{selitems})$, it holds that:
 $\text{Auth}(\mathcal{S}, \mathcal{O}, u, r, \text{read}(\text{ati}, o)).$
- **Clause 6.e.** If either ase_l or ase_r (or both) appear in $\text{PropsInSel}(\text{selitems}) \cup \text{PropsInOn}(\text{exp})$,

then, for every pair of objects $\langle \check{o}_l, \check{o}_r \rangle \in \text{Exec}(\overline{\mathcal{O}}, \text{SELECT } \overline{c}_l_id, \overline{c}_r_id \text{ FROM } \overline{c}_l, \overline{c}_r)$, it holds that:

$\text{Auth}(\mathcal{S}, \mathcal{O}, u, r, \text{read}(asi, o_l, o_r))$.

- **Clause 6.f.** If $\overline{asc_id}$ or any attribute of the association-class's class c' appears in $props = \text{PropsInSel}(selitems) \cup \text{PropsInOn}(exp)$, then
 - If $ase_l \in props$, for every object pair $\langle \check{o}_l, \check{o} \rangle \in \text{Exec}(\overline{\mathcal{O}}, \text{SELECT } ase_l, \overline{asc_id} \text{ FROM } \overline{asc})$, it holds that:
 $\text{Auth}(\mathcal{S}, \mathcal{O}, u, r, \text{read}(asci_l, o_l, o))$.
 - If $ase_r \in props$, for every object pair $\langle \check{o}_r, \check{o} \rangle \in \text{Exec}(\overline{\mathcal{O}}, \text{SELECT } ase_r, \overline{asc_id} \text{ FROM } \overline{asc})$, it holds that:
 $\text{Auth}(\mathcal{S}, \mathcal{O}, u, r, \text{read}(asci_r, o_r, o))$.

Case 7. $q = \text{SELECT } selitems \text{ FROM } \overline{asc} \text{ JOIN } subselect \text{ ON } exp$, where asc is a tuple $\langle c', as, asc_i, asc_e, asc_r, asc_r \rangle$ in ASC , with $as = \langle asi, ase_l, c_l, ase_r, c_r \rangle$. Then, $\text{AuthQuery}(\mathcal{S}, \overline{\mathcal{O}}, u, r, q)$ holds if and only if:

- **Clause 7.a.** $\text{AuthQuery}(\mathcal{S}, \overline{\mathcal{O}}, u, r, subselect)$ holds.
- **Clause 7.b.** For every object $\check{o} \in \text{Exec}(\overline{\mathcal{O}}, \text{SELECT } \overline{asc_id} \text{ FROM } \overline{asc})$, and for every attribute $at = \langle ati, c', t \rangle$, such that $ati \in \text{PropsInOn}(exp)$, it holds that:
 $\text{Auth}(\mathcal{S}, \mathcal{O}, u, r, \text{read}(ati, o))$.
- **Clause 7.c.** For every object $\check{o} \in \text{Exec}(\overline{\mathcal{O}}, \text{SELECT } \overline{asc_id} \text{ FROM } \overline{asc} \text{ JOIN } subselect \text{ ON } exp)$, and for every attribute $at = \langle ati, c', t \rangle$, such that $ati \in \text{PropsInSel}(selitems)$, it holds that:
 $\text{Auth}(\mathcal{S}, \mathcal{O}, u, r, \text{read}(ati, o))$.
- **Clause 7.d.** If either ase_l or ase_r (or both) appear in $\text{PropsInSel}(selitems) \cup \text{PropsInOn}(exp)$, then, for every pair of objects $\langle \check{o}_l, \check{o}_r \rangle \in \text{Exec}(\overline{\mathcal{O}}, \text{SELECT } \overline{c}_l_id, \overline{c}_r_id \text{ FROM } \overline{c}_l, \overline{c}_r)$, it holds that:
 $\text{Auth}(\mathcal{S}, \mathcal{O}, u, r, \text{read}(asi, o_l, o_r))$.
- **Clause 7.e.** If $\overline{asc_id}$ or any attribute of the association-class's class c' appears in $props = \text{PropsInSel}(selitems) \cup \text{PropsInOn}(exp)$, then
 - If $ase_l \in props$, for every object pair $\langle \check{o}_l, \check{o} \rangle \in \text{Exec}(\overline{\mathcal{O}}, \text{SELECT } ase_l, \overline{asc_id} \text{ FROM } \overline{asc} \text{ JOIN } subselect \text{ ON } exp)$, it holds that:
 $\text{Auth}(\mathcal{S}, \mathcal{O}, u, r, \text{read}(asci_l, o_l, o))$.
 - If $ase_r \in props$, for every object pair $\langle \check{o}_r, \check{o} \rangle \in \text{Exec}(\overline{\mathcal{O}}, \text{SELECT } ase_r, \overline{asc_id} \text{ FROM } \overline{asc} \text{ JOIN } subselect \text{ ON } exp)$, it holds that:

$\text{Auth}(\mathcal{S}, \mathcal{O}, u, r, \text{read}(asci_r, o_r, o))$.

We can now give answer to the last questions we posed before:

- Can Bob execute query #5 in every scenario? Yes.
- What about Alice? No.

Case 8. $q = \text{SELECT } selitems \text{ FROM } \overline{c} \text{ JOIN } \overline{as} \text{ ON } exp \text{ WHERE } exp'$, where c is a class in C and as is an association in AS . We reduce this case to the case: $\text{SELECT } selitems \text{ FROM } (\text{SELECT } selitems' \text{ FROM } \overline{c} \text{ JOIN } \overline{as} \text{ ON } exp) \text{ AS TEMP WHERE } exp'$, where $selitems'$ contains $selitems$ and $\text{PropsInWhe}(exp')$.

Case 9. $q = \text{SELECT } selitems \text{ FROM } \overline{c} \text{ JOIN } \overline{asc} \text{ ON } exp \text{ WHERE } exp'$, where c is a class in C and asc is an association-class in ASC . Analogously, We reduce this case to the case: $\text{SELECT } selitems \text{ FROM } (\text{SELECT } selitems' \text{ FROM } \overline{c} \text{ JOIN } \overline{asc} \text{ ON } exp) \text{ AS TEMP WHERE } exp'$, where $selitems'$ contains $selitems$ and $\text{PropsInWhe}(exp')$.

Case 10. $q = \text{SELECT } selitems \text{ FROM } subselect \text{ JOIN } \overline{asc} \text{ ON } exp \text{ WHERE } exp'$, where asc is an association-class in ASC . Analogously, We reduce this case to the case: $\text{SELECT } selitems \text{ FROM } (\text{SELECT } selitems' \text{ FROM } subselect \text{ JOIN } \overline{asc} \text{ ON } exp) \text{ AS TEMP WHERE } exp'$, where $selitems'$ contains $selitems$ and $\text{PropsInWhe}(exp')$.

VI. RELATED WORK

Different languages/formalisms have been proposed in the past for expressing access control policies, each with its own expressive power and limitations [9], [10]. Moreover, other language/formalism have been also proposed for expressing other security requirements, besides access control policies [11]. Notably, UMLsec [12], [13] is a language for annotating UML models with confidentiality and other information-flow requirements. In our work we focus on fine-grained access control (FGAC) policies, which extend standard role-based access control (RBAC) policies [14]. In particular, for FGAC policies, access control decisions depend not only on static information (the assignments of users and permissions to roles), but also on dynamic information (the satisfaction of authorization constraints in the current system state). SecureUML [6] is a language for expressing FGAC policies. Crucially for our work, SecureUML has a well-defined formal semantics, built upon a notion of data models (to specify protected resources) and the

OCL language (to specify authorization constraints). XACML [15] is an extension of XML for defining access control policies that depend on attributes (both of the user and of the protected resources) and variables (of the application environment). [16] has recently proposed XACML4G as an extension of XACML for Graph-structure data, that enables the definition of authorization constraints based on the information contained in the vertices and edges of the graph. Ponder [17], [18] is a language for specifying access control policies using condition-action rules written in a subset of OCL. [19] uses the EMF-IncQuery framework for capturing (fine-grained) access control queries over EMF models. Although SecureUML also uses a query language for specifying fine-grained access control policies, our approach/scope is different. Essentially, while [19] uses the access control queries to provide (to each user) a security-aware (filtered) version/view of the (full) model (in the context of collaborative modeling frameworks), we use the SecureUML models to transform (unsecured) SQL queries into their security-aware versions, which are then executed in the (full) database (in the context of data-centric applications). An in-depth comparison of the expressive power of SecureUML with respect to other languages/formalism (in particular, attribute-based access control languages), is beyond the scope of this paper.

The majority of commercial databases currently support RBAC policies. However, due to the limitations of RBAC as a language for expressing access control policies, a growing number of databases are now providing support for attribute-based access control policies (e.g., SQLServer [20], Oracle [21], Google Firestore [22], MongoDB [23]). Still, none of the current databases can support the FGAC policies (authorization constraints) that we consider in our work. In fact, to the best of our knowledge, [4] was the first attempt of providing a formal characterization of FGAC authorization for SQL queries. Interestingly, [2] has proposed a model-driven approach for enforcing FGAC policies based on [4]. With a different approach, [24] has also attempted to address the challenge of executing securely SQL queries in the context of FGAC policies.

Finally, it is worth recalling the distinction between *Truman* and *non-Truman* models [25] when enforcing (fine-grained) access control policies in (relational) databases. As it is known, the *Truman* model executes a query and then returns the result (possibly partial) after filtering out any unauthorized data. The *non-Truman* model, on the other hand, only returns the result (always

complete) if executing the query did not require to access any unauthorized data. The characterization of FGAC authorization for SQL queries proposed in [4] leans towards the non-Truman model.

VII. CONCLUSION

In this paper we have extended a recently proposed model-based characterization of fine-grained access control (FGAC) [4] in three (inter-related) dimensions. First, we have extended the FGAC policies' underlying data models by considering association-classes. Secondly, we have extended the FGAC policies' security modeling language by considering, as protected resources, the classes and the (explicit and implicit) associations introduced by the association-classes. Thirdly, we have extended the clauses that define whether a user is authorized by an FGAC policy to execute a SQL query, to cover the case of queries retrieving information related to the association-classes. To illustrate our extensions and to demonstrate their applicability, we have provided examples of authorization decisions for SQL queries with respect to FGAC policies.

The next step is to extend the model-driven approach proposed in [2] for enforcing FGAC policies when executing SQL queries. Moreover, we plan to extend [4] to cover the case of SQL aggregation functions and group-by clauses, as well as other types of SQL statements besides select-statements, including insert, update, and delete statements. Another extension of our current work worthwhile exploring is to consider the association-class as a "whole" as a protected resource, so as to be able to specify authorization-constraints that refer at the same time to all the ends (implicit and explicit) of the association-class. In our present work we have opted for considering as "separated" protected resources the association-class's class and the association-class's (explicit and implicit) associations.

Lastly, as long-term project, we plan to develop formal methods to reason about information-leakage when executing SQL queries. These methods will have to take into account both the formal semantics of our security models and the formal semantics of the SQL language.

ACKNOWLEDGMENT

We would like to sincerely thank the anonymous reviewers for their insightful feedback and comments. Their expert evaluation and constructive suggestions greatly contributed to the refinement and quality improvement of the final version of the paper.

REFERENCES

- [1] “The OWASP Top Ten Project,” <https://owasp.org/www-project-top-ten/>, 2021, [Online]. Accessed 10 April 2023.
- [2] H. Nguyen and M. Clavel, “A Model-Driven Approach for Enforcing Fine-Grained Access Control for SQL Queries,” *Springer Nature Computer Science*, vol. 2, no. 5, p. 370, 2021.
- [3] D. A. Basin, M. Clavel, and M. Egea, “A decade of model-driven security,” in *16th ACM Symposium on Access Control Models and Technologies, SACMAT 2011, Innsbruck, Austria, June 15-17, 2011, Proceedings*, R. Breu, J. Crampton, and J. Lobo, Eds. ACM, 2011, pp. 1–10.
- [4] H. Nguyen and M. Clavel, “Model-based Characterization of Fine-Grained Access Control Authorization for SQL Queries,” *Journal of Object Technology*, vol. 19, no. 3, pp. 3:1–13, 2020.
- [5] D. A. Basin, M. Clavel, M. Egea, M. A. G. de Dios, and C. Dania, “A Model-Driven Methodology for Developing Secure Data-Management Applications,” *IEEE Transactions on Software Engineering*, vol. 40, no. 4, pp. 324–337, 2014.
- [6] T. Lodderstedt, D. A. Basin, and J. Doser, “SecureUML: A UML-based modeling language for model-driven security,” in *UML 2002 - The Unified Modeling Language, 5th International Conference, Dresden, Germany, September 30 - October 4, 2002, Proceedings*, ser. Lecture Notes in Computer Science, J. Jézéquel, H. Hußmann, and S. Cook, Eds., vol. 2460. Springer, 2002, pp. 426–441.
- [7] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn, and R. Chandramouli, “Proposed NIST Standard for Role-Based Access Control,” *ACM Transactions on Information and System Security*, vol. 4, no. 3, pp. 224–274, Aug. 2001.
- [8] “Object Constraint Language specification version 2.4,” <https://www.omg.org/spec/OC/L/>, Object Management Group, Tech. Rep., February 2014, [Online]. Accessed 10 April 2023.
- [9] A. K. Y. S. Mohamed, D. Auer, D. Hofer, and J. Küng, “Authorization strategies and classification of access control models,” in *Future Data and Security Engineering - 8th International Conference, FDSE 2021, Virtual Event, November 24-26, 2021, Proceedings*, ser. Lecture Notes in Computer Science, T. K. Dang, J. Küng, T. M. Chung, and M. Takizawa, Eds., vol. 13076. Springer, 2021, pp. 155–174. [Online]. Available: https://doi.org/10.1007/978-3-030-91387-8_11
- [10] “NISTIR 7874: Guidelines for Access Control System Evaluation Metrics,” <https://nvlpubs.nist.gov/nistpubs/ir/2012/NIST.IR.7874.pdf>, 2012, [Online]. Accessed 10 April 2023.
- [11] L. Lucio, Q. Zhang, P. H. Nguyen, M. Amrani, J. Klein, H. Vangheluwe, and Y. L. Traon, “Advances in model-driven security,” *Journal of Advanced Computing*, vol. 93, pp. 103–152, 2014. [Online]. Available: <https://doi.org/10.1016/B978-0-12-800162-2.00003-8>
- [12] J. Jürjens, “UMLsec: Extending UML for secure systems development,” in *The Unified Modeling Language (UML)*, ser. LNCS, J. Jézéquel, H. Hußmann, and S. Cook, Eds., vol. 2460. Springer, 2002, pp. 412–425. [Online]. Available: https://doi.org/10.1007/3-540-45800-X_32
- [13] S. H. Houmb, S. Islam, E. Knauss, J. Jürjens, and K. Schneider, “Eliciting security requirements and tracing them to design: an integration of common criteria, heuristics, and UMLsec,” *Requir. Eng.*, vol. 15, no. 1, pp. 63–93, 2010. [Online]. Available: <https://doi.org/10.1007/s00766-009-0093-9>
- [14] “INCITS 359-2012: Information technology - Role Based Access Control,” https://standards.incits.org/apps/group_public/project/details.php?project_id=1658, 2019, [Online]. Accessed 10 April 2023.
- [15] E. Rissanen, “eXtensible Access Control Markup Language (XACML) Version 3.0,” <http://docs.oasis-open.org/xacml/3.0/>, OASIS, Tech. Rep., 2013, [Online]. Accessed 10 April 2023.
- [16] A. K. Y. S. Mohamed, D. Auer, D. Hofer, and J. Küng, “Extended Authorization Policy for Graph-Structured Data,” *SN Comput. Sci.*, vol. 2, no. 5, p. 351, 2021. [Online]. Available: <https://doi.org/10.1007/s42979-021-00684-8>
- [17] N. Damianou, N. Dulay, E. Lupu, and M. Sloman, “The Ponder Policy Specification Language,” in *Policies for Distributed Systems and Networks*, ser. LNCS, M. Sloman, J. Lobo, and E. Lupu, Eds., vol. 1995. Springer, 2001, pp. 18–38. [Online]. Available: https://doi.org/10.1007/3-540-44569-2_2
- [18] N. Damianou, N. Dulay, E. Lupu, M. Sloman, and T. Tonouchi, “Tools for domain-based policy management of distributed systems,” in *Management Solutions for the New Communications World, 8th IEEE/IFIP Network Operations and Management Symposium, NOMS 2002, Florence, Italy, April 15-19, 2002. Proceedings*, R. Stadler and M. Ulema, Eds. IEEE, 2002, pp. 203–217. [Online]. Available: <https://doi.org/10.1109/NOMS.2002.1015565>
- [19] G. Bergmann, C. Debrececi, I. Ráth, and D. Varró, “Query-based access control for secure collaborative modeling using bidirectional transformations,” in *Proceedings of the ACM/IEEE 19th International Conference on Model Driven Engineering Languages and Systems, Saint-Malo, France, October 2-7, 2016*, B. Baudry and B. Combemale, Eds. ACM, 2016, pp. 351–361. [Online]. Available: <http://dl.acm.org/citation.cfm?id=2976793>
- [20] “Row-Level Security,” <https://learn.microsoft.com/en-us/sql/relational-databases/security/row-level-security>, Microsoft Corporation, Tech. Rep., 2023, [Online]. Accessed 10 April 2023.
- [21] K. Browder and M. A. Davidson, “The virtual private database in Oracle9iR2,” Oracle Corporation, Tech. Rep., 2002, <https://www.cgisecurity.com/database/oracle/pdf/VPD9ir2twp.pdf>.
- [22] “Writing conditions for Cloud Firestore Security Rules | Firebase documentation,” <https://firebase.google.com/docs/firestore/security/rules-conditions>, 2019, [Online]. Accessed 10 April 2023.
- [23] “Rule Expressions MongoDB Realm,” n.d., <https://docs.mongodb.com/realm/rules/expressions/>.
- [24] A. Mehta, E. Elnikety, K. Harvey, D. Garg, and P. Druschel, “Qapla: Policy compliance for database-backed systems,” in *26th USENIX Security Symposium, USENIX Security 2017, Vancouver, BC, Canada, August 16-18, 2017*, E. Kirda and T. Ristenpart, Eds. USENIX Association, 2017, pp. 1463–1479.
- [25] S. Rizvi, A. Mendelzon, S. Sudarshan, and P. Roy, “Extending query rewriting techniques for fine-grained access control,” in *Proceedings of the 2004 ACM SIGMOD International Conference on Management of Data*, ser. SIGMOD ’04. New York, NY, USA: Association for Computing Machinery, 2004, pp. 551–562. [Online]. Available: <https://doi.org/10.1145/1007568.1007631>